

Memoria de Práctica 6

Servidores Web Apache y Proxy Nginx

Administración de Servidores

19 de mayo de 2026

Resumen

En esta memoria se explica la configuración que forma parte de la Práctica 6, junto con las decisiones técnicas aplicadas y las pruebas que permiten comprobar su funcionamiento. El proyecto consiste en desplegar un servidor web Apache con dos zonas diferenciadas, una pública y otra destinada a la intranet. Además, se integra el servicio web con el servidor FTP configurado en la práctica anterior, se aplican restricciones de acceso sobre determinados recursos y se personalizan las páginas de error del servidor. Finalmente, se documenta la implementación de un proxy inverso con Nginx.

Índice

1. Apartado A: Despliegue del Servidor Web Apache	3
1.1. Arquitectura y Despliegue del Contenedor Web	3
1.2. Configuración de Resolución DNS	3
1.3. Integración con el Servidor FTP	3
1.4. Configuración de Sitios Web (VirtualHosts)	4
1.4.1. Sitio Público (<code>www.empresa.local</code>)	4
1.4.2. Sitio Intranet (<code>intranet.empresa.local</code>)	4
1.5. Pruebas de Verificación (Apartado A)	4
2. Apartado B: Políticas de Restricción y Control de Peticiones	8
2.1. Bloqueo de Contenido Dinámico (.php)	8
2.2. Páginas de Error Personalizadas	10
3. Bonus: Nginx como Proxy Inverso	13
3.1. Configuración de Nginx e Integración DNS	13
3.2. Políticas de Firewall (Iptables)	13
3.3. Pruebas y Verificación del Perfil VPN	14
4. Bonus 2: Implementación y Análisis de Logs Avanzados	16
4.1. Configuración del Servidor	16
4.2. Análisis de Escenarios en Logs	16
4.3. Evidencias de Funcionamiento	16

1. Apartado A: Despliegue del Servidor Web Apache

Según la arquitectura planteada para la práctica, el servidor Apache se sitúa dentro de la Red de Desarrollo. El servicio HTTP escucha en el puerto 80 y ofrece dos accesos separados: uno público, pensado como página principal de la empresa, y otro privado, destinado a la intranet y limitado a los clientes que acceden mediante VPN.

1.1. Arquitectura y Despliegue del Contenedor Web

Para integrar el servidor web en la infraestructura Docker, se aplican los siguientes cambios:

- **Modificación en docker compose:** Se añade el servicio `apache_web` dentro del archivo `stacks/development/compose.yml`.
- **Imagen base:** Se utiliza una imagen personalizada basada en `httpd:2.4`. En el Dockerfile se habilitan de forma automática los módulos necesarios, como `vhost_alias`, y se incluye la configuración externa mediante el fichero `httpd-vhosts.conf`.
- **Red e IP:** El contenedor se conecta a la red interna `development_net` y utiliza la dirección IP fija `172.40.0.10`.

1.2. Configuración de Resolución DNS

Para que el acceso a los sitios web sea más cómodo y no dependa directamente de la dirección IP del contenedor, se configura la resolución de nombres en el servidor Bind9 de la red de servicios:

- **Registros DNS:** En el archivo de zona `db.empresa.local` se incluyen dos nuevos registros de tipo A:
 - `www IN A 172.40.0.10`
 - `intranet IN A 172.40.0.10`
- **Resolución en contenedores de prueba:** Los contenedores de la red de desarrollo utilizan el DNS interno (`dns: 172.20.0.10`), lo que permite realizar pruebas por nombre de dominio directamente desde la consola.

1.3. Integración con el Servidor FTP

Apache también publica por HTTP los archivos que se encuentran en la carpeta “público” del servidor FTP configurado en la práctica anterior. Para ello, ambos servicios comparten un mismo volumen:

- **Volumen compartido:** Se utiliza un Docker Volume llamado `publico_data`.
- El servidor FTP monta este volumen en `/data/publico`, que actúa como directorio para el grupo `ftp`.
- El servidor Web monta el mismo volumen en modo solo lectura (`ro`) dentro de `/var/www/ftp-publico`.

Con esta configuración, los archivos gestionados desde el FTP también quedan disponibles desde la web. Apache solo tiene permisos de lectura sobre ese contenido, por lo que no puede modificar ni eliminar los ficheros publicados.

1.4. Configuración de Sitios Web (VirtualHosts)

La separación entre los dos dominios se realiza mediante el fichero `httpd-vhosts.conf`. En este archivo se definen los dos *VirtualHosts*, cada uno con su propio directorio raíz y con las restricciones que corresponden a su función.

1.4.1. Sitio Público (`www.empresa.local`)

El `DocumentRoot` del sitio público apunta a `/var/www/www`. En este directorio se encuentra la página principal `index.html`, que muestra el logotipo de *TechSolutions*, los nombres de los integrantes del grupo y un enlace hacia los contenidos publicados desde el FTP.

El acceso a la carpeta `/publico` se protege mediante autenticación básica de Apache. Para ello, se utiliza un archivo `.htpasswd` con un hash SHA-1 asociado al usuario `documentos`, cuya contraseña es `GraTiS!`. De esta manera, el contenido compartido desde el FTP se puede consultar desde el navegador, pero solo después de introducir credenciales válidas.

1.4.2. Sitio Intranet (`intranet.empresa.local`)

El sitio de intranet utiliza como `DocumentRoot` el directorio `/var/www/intranet`. En él se aloja una página sencilla que identifica esta zona como un espacio restringido.

La protección de este *VirtualHost* se basa en la directiva `Require ip 172.10.0.0/24`, que corresponde a la subred asignada por pfSense a los clientes VPN. Por tanto, solo los equipos conectados a través de la VPN pueden acceder a la intranet. Cualquier petición realizada desde otra red recibe una respuesta 403 Forbidden.

1.5. Pruebas de Verificación (Apartado A)

Para comprobar que la configuración funciona correctamente, se realizan pruebas tanto desde el navegador como desde la consola de comandos.

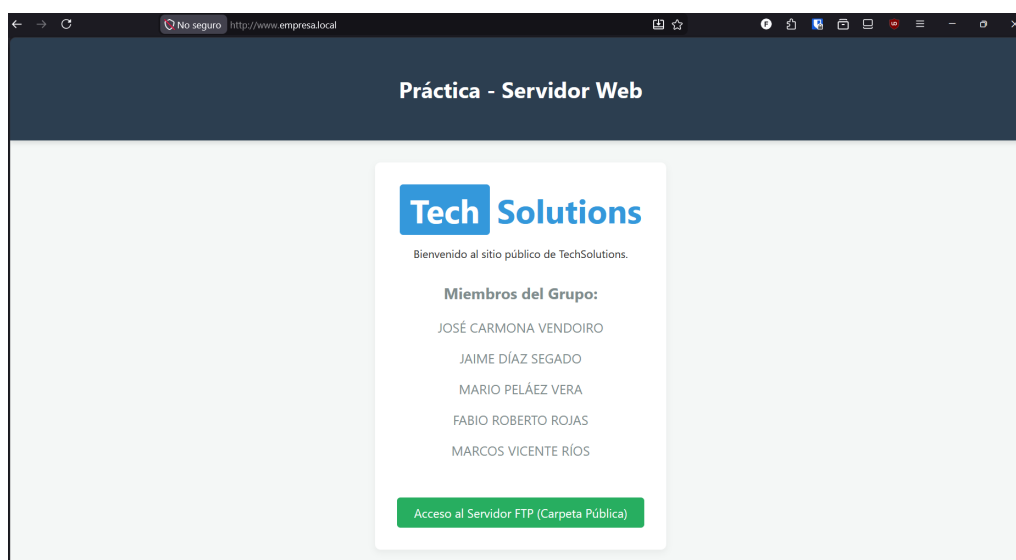


Figura 1: Acceso a la página principal de la empresa mediante `www.empresa.local`.

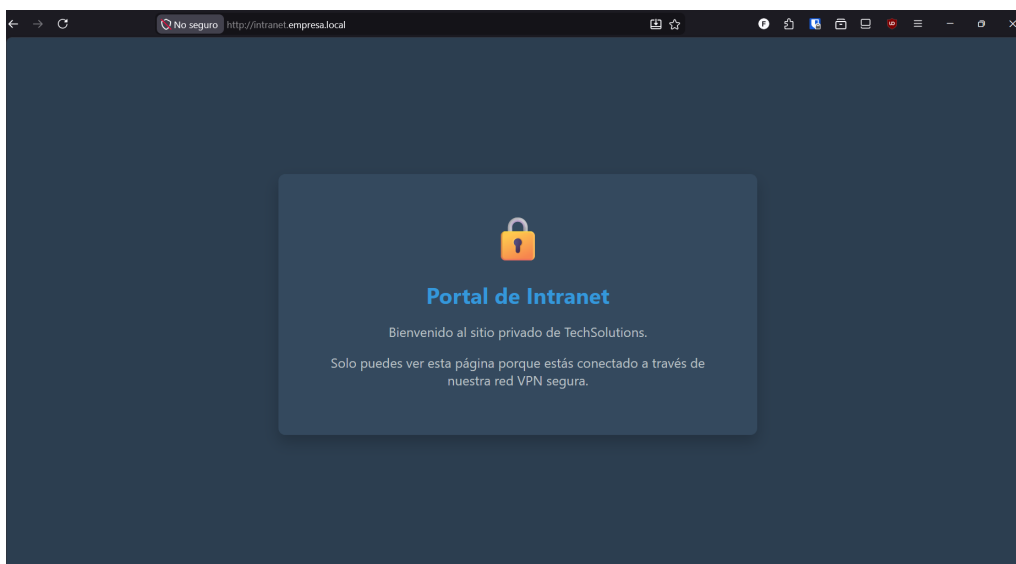


Figura 2: Acceso correcto a la intranet `intranet.empresa.local` desde un cliente automatizado conectado por VPN.

Al entrar en el enlace de la carpeta pública, el navegador solicita usuario y contraseña mediante autenticación básica:

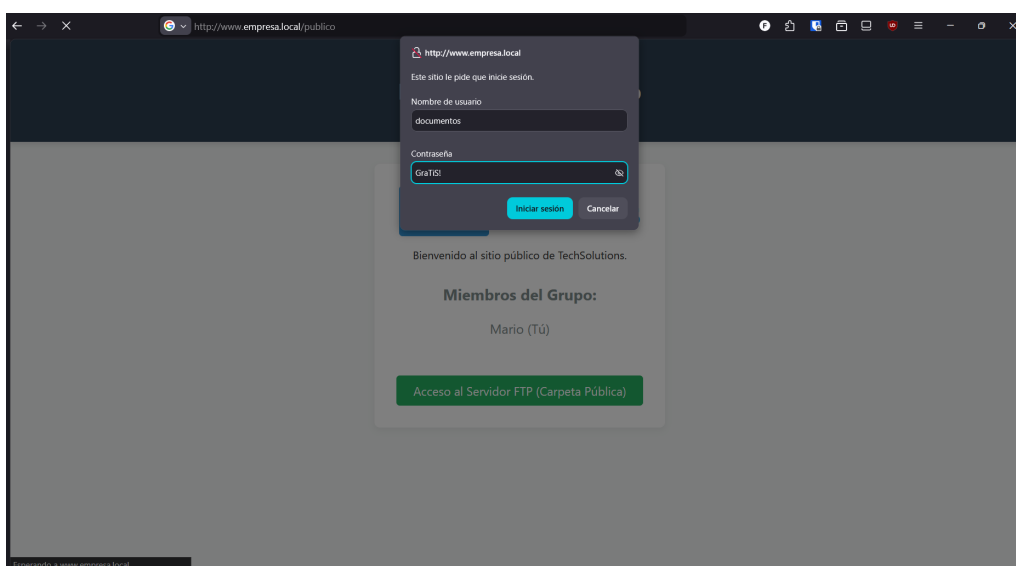


Figura 3: Solicitud de credenciales al intentar acceder al directorio `/publico`.

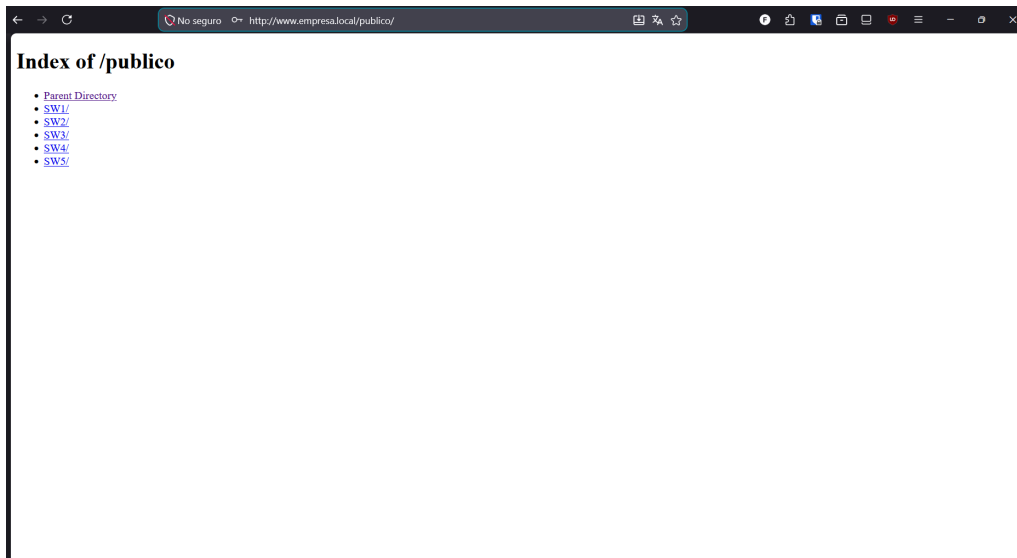


Figura 4: Listado de archivos del directorio FTP mostrado por Apache después de introducir credenciales válidas.

También se comprueba el comportamiento del servidor mediante comandos `curl`. Estas pruebas permiten verificar las respuestas HTTP sin depender únicamente del navegador:

```
root@dev-ssh:/# curl -s http://www.empresa.local/publico
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>401 Unauthorized</title>
</head><body>
<h1>Unauthorized</h1>
<p>This server could not verify that you
are authorized to access the document
requested. Either you supplied the wrong
credentials (e.g., bad password), or your
browser doesn't understand how to supply
the credentials required.</p>
</body></html>
```

Figura 5: Prueba con `curl` sin credenciales contra `/publico`, donde el servidor responde con un error 401 Unauthorized.

```
root@dev-ssh:/# curl -s -u documentos:GratIS! http://www.empresa.local/publico/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>Index of /publico</title>
</head>
<body>
<h1>Index of /publico</h1>
<ul><li><a href="/"> Parent Directory</a></li>
<li><a href="SW1/"> SW1</a></li>
<li><a href="SW2/"> SW2</a></li>
<li><a href="SW3/"> SW3</a></li>
<li><a href="SW4/"> SW4</a></li>
<li><a href="SW5/"> SW5</a></li>
</ul>
</body></html>
```

Figura 6: Prueba con `curl` utilizando credenciales válidas, en la que se muestra correctamente el índice de la carpeta pública.

```
root@dev-ssh:/# curl -s http://intranet.empresa.local/publico/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
</body></html>
```

Figura 7: Prueba con `curl` al intentar acceder a la carpeta pública desde la URL de la intranet, comprobando el bloqueo mediante 403 Forbidden.

2. Apartado B: Políticas de Restricción y Control de Peticiones

En este apartado se añaden medidas de control sobre el servidor web con la finalidad de limitar el acceso a ciertos tipos de archivos y mejorar la respuesta que recibe el usuario cuando se produce un error, por ejemplo al acceder a una zona no permitida o a una ruta que no existe.

2.1. Bloqueo de Contenido Dinámico (.php)

La práctica indica que el servidor no debe ejecutar contenido dinámico. Por este motivo, cualquier petición a un archivo con extensión `.php` queda bloqueada. Para aplicarlo, se incorpora la siguiente regla en la configuración global de Apache:

```
1 <FilesMatch "\.php$">
2     Require all denied
3 </FilesMatch>
```

Listing 1: Bloqueo de ejecución de archivos PHP

Esta regla utiliza una expresión regular para localizar todos los archivos cuyo nombre termina en `.php`. Cuando Apache recibe una petición de este tipo, deniega el acceso de forma directa y devuelve un código de error personalizado. Así se evita que el servidor procese o muestre archivos PHP, manteniendo el sitio como un entorno de contenido estático.


```

jaime@minty:~$ curl -i http://localhost/.htpasswd
HTTP/1.1 403 Forbidden
Date: Wed, 06 May 2026 17:24:20 GMT
Server: Apache/2.4.67 (Unix)
Last-Modified: Wed, 06 May 2026 15:44:19 GMT
ETag: "5e5-651280971da52"
Accept-Ranges: bytes
Content-Length: 1509
Content-Type: text/html

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>403 - Acceso no autorizado</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: Arial, sans-serif;
      background-color: #1a1a2e;
      color: #e0e0e0;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
    }
    .container {
      text-align: center;
      padding: 2rem;
    }
    .code {
      font-size: 6rem;
      font-weight: bold;
      color: #e63946;
    }
    h1 {
      font-size: 1.8rem;
      margin: 0.5rem 0 1rem;
      color: #f1faee;
    }
    p {
      font-size: 1rem;
      color: #a8a8b3;
      margin-bottom: 2rem;

```

Figura 8: Intento de acceso a contenido no autorizado mediante curl. Apache rechaza la petición con un error HTTP 403 Forbidden y devuelve el HTML correspondiente a la página de error personalizada.

```

jaime@minty:~$ curl -i http://localhost/pagina-falsa
HTTP/1.1 404 Not Found
Date: Wed, 06 May 2026 16:23:54 GMT
Server: Apache/2.4.67 (Unix)
Last-Modified: Wed, 06 May 2026 15:44:21 GMT
ETag: "5f3-651280990fb51"
Accept-Ranges: bytes
Content-Length: 1523
Content-Type: text/html

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>404 - Recurso no encontrado</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: Arial, sans-serif;
      background-color: #1a1a2e;
      color: #e0e0e0;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
    }
    .container {
      text-align: center;
      padding: 2rem;
    }
    .code {
      font-size: 6rem;
      font-weight: bold;
      color: #f4a261;
    }
    h1 {
      font-size: 1.8rem;
      margin: 0.5rem 0 1rem;
      color: #f1faee;
    }
    p {
      font-size: 1rem;
      color: #a8a8b3;
      margin-bottom: 2rem;
    }
  </style>
</head>
<body>
  <h1>404</h1>
  <p>Recurso no encontrado</p>
</body>
</html>

```

Figura 9: Intento de acceso a contenido inexistente mostrando el error 404 Not Found.

```

jaime@minty:~/AS-2025$ curl -i http://localhost/index.php
HTTP/1.1 302 Found
Date: Wed, 06 May 2026 17:38:40 GMT
Server: Apache/2.4.67 (Unix)
Location: http://localhost/errores/php-bloqueado.html
Content-Length: 267
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>302 Found</title>
</head><body>
<h1>Found</h1>
<p>The document has moved <a href="http://localhost/errores/php-bloqueado.html">here</a>.</p>
</body></html>

```

Figura 10: Acceso mediante curl a contenido dinámico mostrando el error personalizado.

2.2. Páginas de Error Personalizadas

El servidor también utiliza páginas de error personalizadas para no mostrar las respuestas predeterminadas de Apache. Esto permite dar una apariencia más cuidada al sitio y facilita que el usuario entienda mejor qué ocurre cuando no puede acceder a un recurso.

Para configurar este comportamiento, se añaden directivas **ErrorDocument** que apuntan a archivos HTML estáticos preparados para cada caso. En concreto, se definen páginas específicas para los errores de acceso no permitido y para los recursos no encontrados:

```

1 RedirectMatch 302 .php$ /errores/php-bloqueado.html
2
3 ErrorDocument 403 /errores/403.html

```

```
4 ErrorDocument 404 /errores/404.html
```

Listing 2: Configuración de ErrorDocument en VirtualHosts

Después de aplicar esta configuración, se fuerzan varios errores 403 y 404 para comprobar que Apache carga las páginas personalizadas en lugar de sus mensajes por defecto.

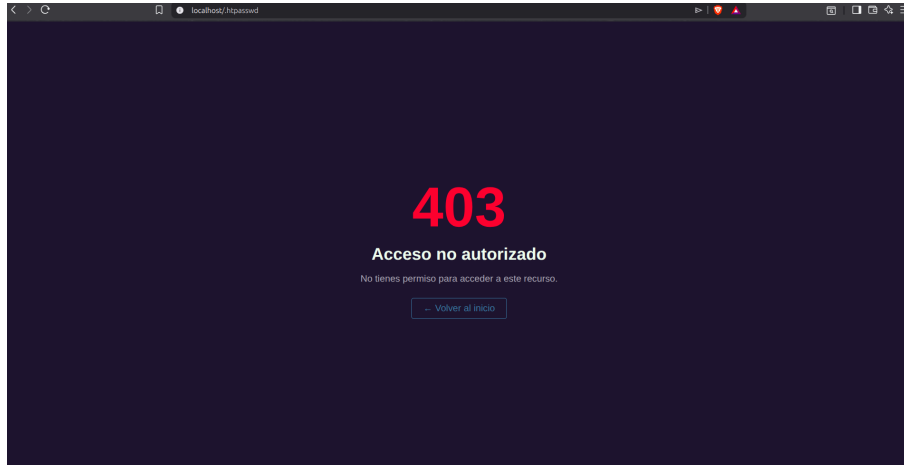


Figura 11: Prueba en navegador de una respuesta personalizada ante un error HTTP 403, al acceder a un recurso no autorizado.

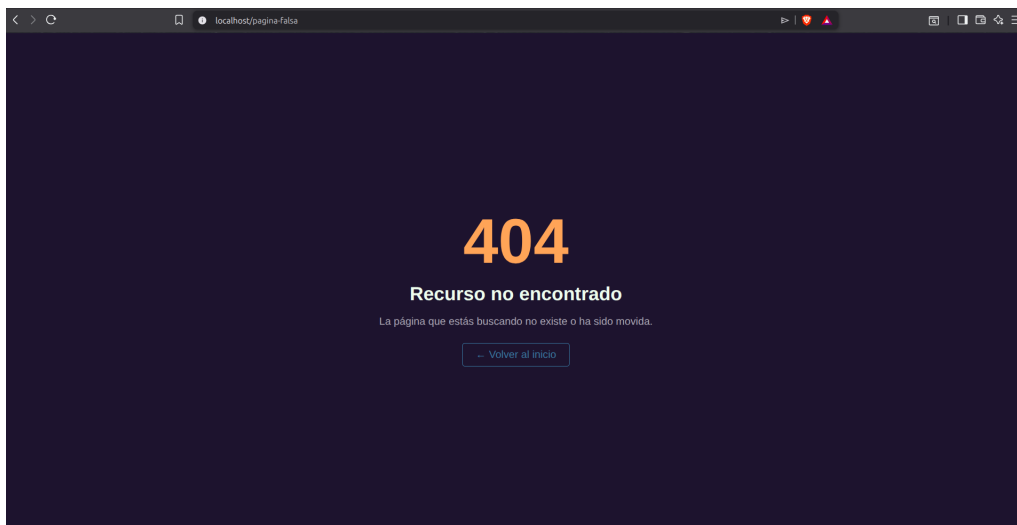


Figura 12: Prueba de acceso a una ruta inexistente, /pagina-falsa, mostrando la página personalizada del error 404 Recurso no encontrado.

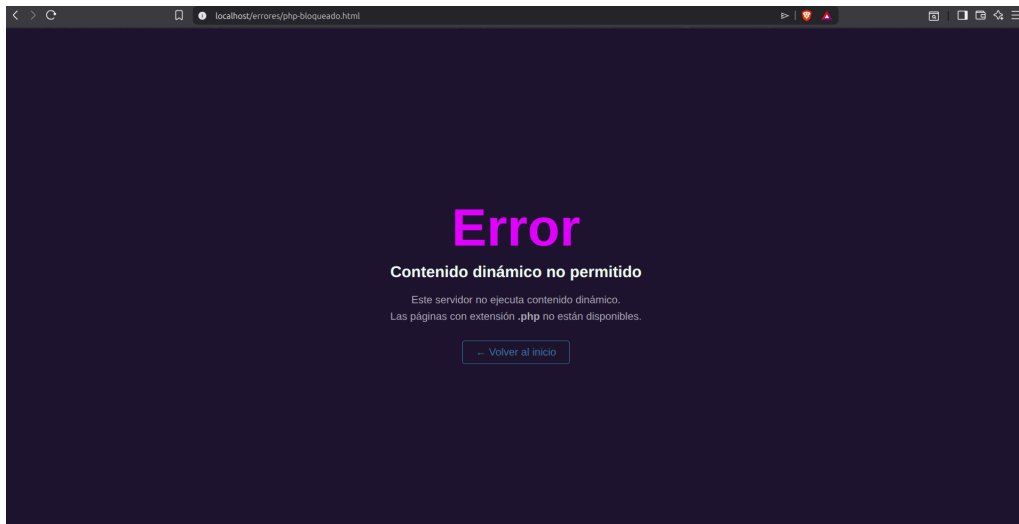


Figura 13: Prueba de acceso al intentar acceder a un recurso dinámico, mostrando la página personalizada del error.

3. Bonus: Nginx como Proxy Inverso

En esta sección adicional se documenta la integración de un servidor web Nginx actuando como proxy inverso. El objetivo es que las peticiones destinadas a la Intranet de la empresa sean recibidas inicialmente por Nginx en la red de producción, y posteriormente redirigidas al servidor Apache de la red de desarrollo.

El plan seguido consta de cuatro fases principales:

1. Configurar el reverse proxy en Nginx hacia el backend Apache.
2. Ajustar el DNS interno para resolver `intranet.empresa.local` hacia el proxy.
3. Restringir el acceso por firewall a usuarios conectados por VPN.
4. Regenerar perfiles VPN y validar conectividad con pruebas desde el cliente.

3.1. Configuración de Nginx e Integración DNS

Se crea un bloque de servidor (*virtual host*) para `intranet.empresa.local` interceptando el puerto 80 y derivando el tráfico al contenedor de Apache (172.40.0.10):

```
1 server {
2     listen 80;
3     server_name intranet.empresa.local;
4
5     location / {
6         proxy_pass http://172.40.0.10;
7         proxy_http_version 1.1;
8         proxy_set_header Host $host;
9         proxy_set_header X-Real-IP $remote_addr;
10        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
11        proxy_set_header X-Forwarded-Proto $scheme;
12        proxy_set_header Connection "";
13    }
14 }
```

Listing 3: Configuración del proxy inverso (intranet.conf)

Simultáneamente, se ajusta el servidor Bind9 editando el archivo de zona `db.empresa.local` para que las resoluciones del nombre `intranet` apunten a la IP del Nginx en la red de producción (172.30.0.12), recargando el servicio DNS a continuación.

3.2. Políticas de Firewall (Iptables)

Para garantizar que únicamente los usuarios de la VPN puedan acceder al proxy, se aplican reglas estrictas de filtrado en el firewall del nodo. Se configura iptables para permitir el tráfico HTTP/HTTPS desde la red VPN (172.10.0.0/24) y la propia subred de producción hacia el Nginx, denegando el resto de conexiones. Además, se permitió expresamente el reenvío (*forwarding*) desde el proxy hacia el backend Apache:

```
1 #####
2 # NGINX PROXY -> APACHE
3 #####
4 # Permitir HTTP/HTTPS desde proxy (172.30.0.12) a apache (172.40.0.10)
5 -A FORWARD -p tcp -s 172.30.0.12 -d 172.40.0.10 --dport 80 -j ACCEPT
6 -A FORWARD -p tcp -s 172.30.0.12 -d 172.40.0.10 --dport 443 -j ACCEPT
7
8 #####
9 # NGINX (SOLO VPN O MISMA SUBRED)
10 #####
11 # Permitir acceso HTTP/HTTPS al nginx desde VPN y produccion
```

```

12 -A FORWARD -p tcp -s 172.10.0.0/24 -d 172.30.0.12 --dport 80 -j ACCEPT
13 -A FORWARD -p tcp -s 172.10.0.0/24 -d 172.30.0.12 --dport 443 -j ACCEPT
14 -A FORWARD -p tcp -s 172.30.0.0/24 -d 172.30.0.12 --dport 80 -j ACCEPT
15 -A FORWARD -p tcp -s 172.30.0.0/24 -d 172.30.0.12 --dport 443 -j ACCEPT
16
17 # Bloquear HTTP/HTTPS al nginx desde otras redes
18 -A FORWARD -p tcp -d 172.30.0.12 --dport 80 -j DROP
19 -A FORWARD -p tcp -d 172.30.0.12 --dport 443 -j DROP
20
21 # VPN -> intranet DNS (172.30.0.10) hacia nginx real (172.30.0.12)
22 -A PREROUTING -i wg0 -p tcp -d 172.30.0.10 --dport 80 -j DNAT --to-
    destination 172.30.0.12:80
23 -A PREROUTING -i wg0 -p tcp -d 172.30.0.10 --dport 443 -j DNAT --to-
    destination 172.30.0.12:443

```

Listing 4: Reglas Iptables (iptables.rules)

3.3. Pruebas y Verificación del Perfil VPN

Finalmente, se regeneran los perfiles de WireGuard y se procede a validar la conectividad desde un cliente. Como se muestra en las siguientes figuras, la conexión se establece correctamente y la página de la Intranet responde a través del túnel VPN utilizando el entorno proxy configurado.

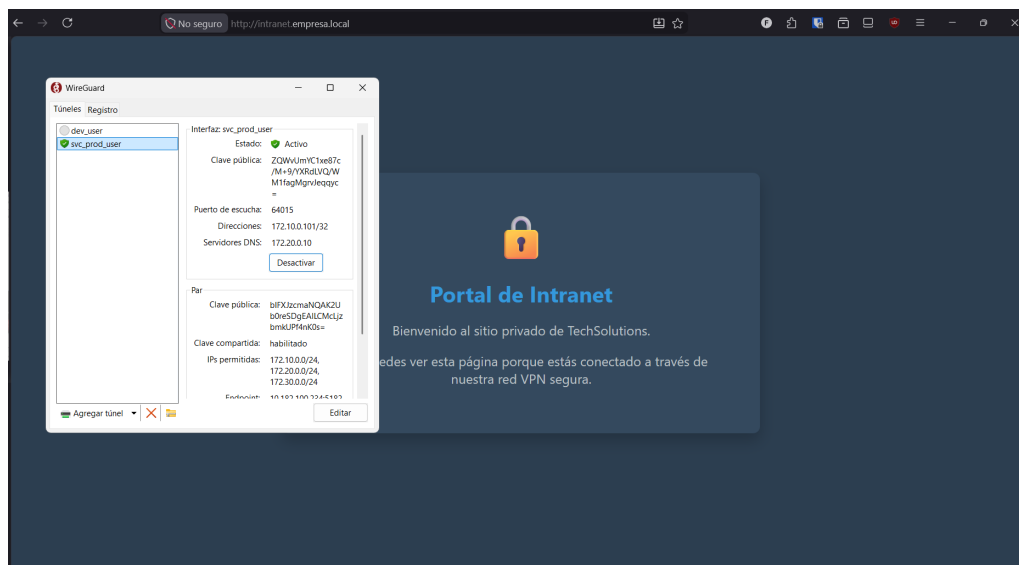


Figura 14: Validación de acceso a la intranet utilizando el perfil VPN del usuario de desarrollo (dev_user).

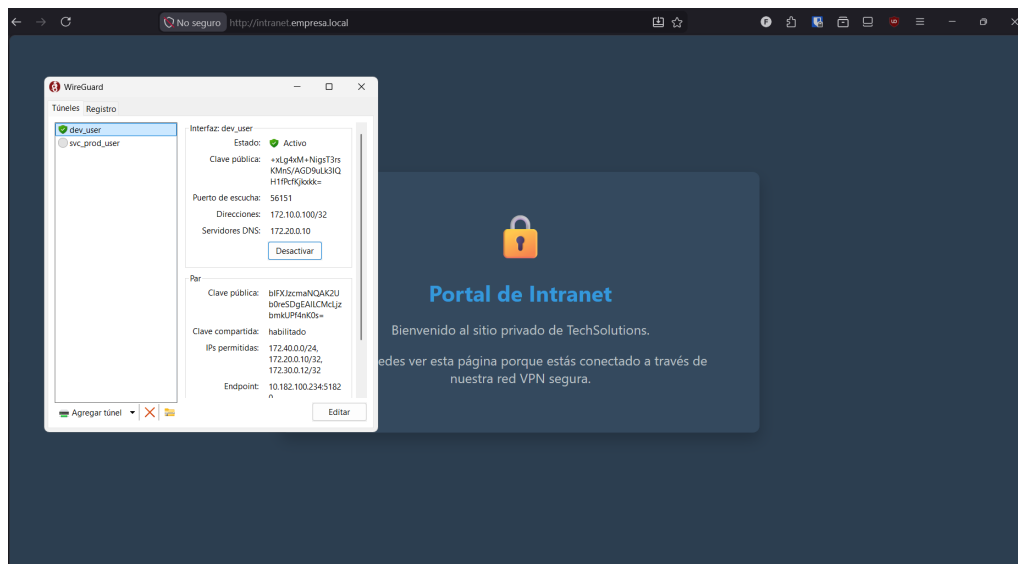


Figura 15: Validación de acceso a la intranet utilizando el perfil VPN del usuario de servicios de producción (`svc_prod.user`).

4. Bonus 2: Implementación y Análisis de Logs Avanzados

Para implementar este mecanismo sin recurrir a lenguajes dinámicos, la estrategia se fundamenta en el módulo `mod.usertrack` de Apache, encargado de generar y asignar cookies de forma automática. Paralelamente, se ajustó el proxy inverso Nginx para inyectar la IP real del cliente en las cabeceras HTTP, permitiendo su correcto registro en el servidor final.

4.1. Configuración del Servidor

Tras habilitar el módulo de rastreo, se modificó el *VirtualHost* de la intranet para emitir la cookie `IdentificadorCliente`. Además, se adaptó la directiva `LogFormat` para capturar simultáneamente la IP del proxy (`%h`), la IP real inyectada por Nginx (`X-Forwarded-For`) y el valor de la cookie interceptada en la petición.

```
1 CookieTracking on
2 CookieName IdentificadorCliente
3 CookieExpires "30 days"
4
5 LogFormat "%h %l %u %t \"%r\" %>s %b IP_Real: %{X-Forwarded-For}i Cookie:
6           %{IdentificadorCliente}C" proxy_custom_log
7 CustomLog /var/log/apache2/intranet_access.log proxy_custom_log
```

Listing 5: Configuración de usertrack y logs en Apache

4.2. Análisis de Escenarios en Logs

De acuerdo con la nueva directiva de logs, el análisis de los accesos revela los siguientes indicadores para cada escenario:

- **Acceso mediante VPN:** La variable `IP_Real` muestra una dirección perteneciente a la subred VPN. En la primera visita, el campo cookie registra un guion (-) y Apache emite el identificador en su respuesta.
- **Intento de acceso sin VPN:** La petición es bloqueada por el firewall. Si alcanzara el proxy, el indicador `IP_Real` delataría inmediatamente una IP pública o local no autorizada.
- **Acceso repetido (mismo navegador):** Se observan múltiples líneas de log en distintos momentos con la misma `IP_Real` y exactamente el mismo valor en el campo `Cookie`.
- **Acceso desde distintos navegadores:** Las peticiones comparten la misma `IP_Real`, pero muestran distintos valores en el campo `Cookie`, ya que cada navegador posee un almacén aislado.
- **Acceso en modo incógnito:** La primera petición muestra `Cookie: -`. Las peticiones sucesivas se rastrean temporalmente, pero al cerrar y reabrir la ventana, la cookie se destruye, generando un rastro totalmente nuevo y desvinculado.

4.3. Evidencias de Funcionamiento

Las siguientes pruebas realizadas en consola validan el ciclo completo de la cookie y su registro.

Se ejecuta una petición inicial limpia para comprobar cómo Apache genera el identificador:


```

PS C:\Users\symbiontz> curl.exe -v -H "Host: intranet.empresa.local" http://localhost:8080/
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8080...
* Established connection to localhost (::1 port 8080) from ::1 port 53074
* using HTTP/1.x
> GET / HTTP/1.1
> Host: intranet.empresa.local
> User-Agent: curl/8.19.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.29.8
< Date: Mon, 18 May 2026 23:24:40 GMT
< Content-Type: text/html
< Content-Length: 1442
< Connection: keep-alive
< Set-Cookie: IdentificadorCliente=66aea72d.6521fdddf0bb4; path=/; max-age=2592000
< Last-Modified: Wed, 06 May 2026 16:43:55 GMT
< ETag: "5a2-65128de95c8b7"
< Accept-Ranges: bytes

```

Figura 16: Petición inicial: Apache responde con la cabecera Set-Cookie.

Para simular el comportamiento de un usuario recurrente, se realiza una nueva petición inyectando manualmente un valor de cookie:

```

1 curl.exe -v -H "Host: intranet.empresa.local" -H "Cookie:
  IdentificadorCliente=EL_VALOR_DE_TU_COOKIE" http://localhost:8080/

```

```

PS C:\Users\symbiontz> curl.exe -v -H "Host: intranet.empresa.local" -H "Cookie: IdentificadorCliente=EL_VALOR_DE_TU_COOKIE" http://localhost:8080/
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8080...
* Established connection to localhost (::1 port 8080) from ::1 port 53069
* using HTTP/1.x
> GET / HTTP/1.1
> Host: intranet.empresa.local
> User-Agent: curl/8.19.0
> Accept: */*
> Cookie: IdentificadorCliente=EL_VALOR_DE_TU_COOKIE
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.29.8
< Date: Mon, 18 May 2026 23:23:38 GMT
< Content-Type: text/html
< Content-Length: 1442
< Connection: keep-alive
< Last-Modified: Wed, 06 May 2026 16:43:55 GMT
< ETag: "5a2-65128de95c8b7"
< Accept-Ranges: bytes

```

Figura 17: Simulación de un cliente recurrente inyectando una cookie existente.

La monitorización en tiempo real de los logs confirma que el servidor extrae y registra correctamente la IP real de origen y el valor exacto de la cookie inyectada en la petición anterior:

```

symbiontz@DESKTOP-NIU59S:/mnt/c/Users/symbiontz/projects/AS-2025$ docker logs --tail 10 dev_apache_web
[Mon May 18 23:23:23.275791 2026] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.67 (Unix) configured -- resuming normal operations
[Mon May 18 23:23:23.276058 2026] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGROUND'
172.40.0.2 - - [18/May/2026:23:23:38 +0000] "GET / HTTP/1.1" 200 1442 "-" "curl/8.19.0" IP_Real: 172.30.0.1 Cookie: EL_VALOR_DE_TU_COOKIE

```

Figura 18: Logs de Apache registrando la IP real de origen y el valor de la cookie.